Writing a Basic Framebuffer Driver

This article deals with the basic structure of a framebuffer and will interest those who know how to write a character device driver. The author has tried to simplify the topic as much as possible so as to make it accessible to more readers.

A framebuffer driver is an intermediate layer in Linux, which hides the complexities of the underlying video device from the user space applications. From the point of view of the user space, if the display device needs to be accessed for reading or writing, then only the framebuffer device such as */dev/fb0* has to be accessed. If you have more than one video card in your computer, then each will be assigned separate device nodes like */dev/fb0.. /dev/fb1.. /dev/fbX* (X being the minor number of the device).

We also have many different ways to access a framebuffer. To get a snapshot of your screen, you can just use the *cp* command, as follows:

```
cp /dev/fb0 myscreen
```

From a programmer's point of view, you can read and write to this device, the main use being *mmap*. You can map the video memory in the user space memory, and then you can control each and every pixel of your screen. We also have a set of ioctl calls to get and set different parameters of the video hardware. A few ioctl commands include *FBIOGET_VSCREENINFO* and *FBIOPUT_VSCREENINFO*. As you may have guessed, the *FBIOGET_VSCREENINFO* call will ask the hardware about the screen information, and *FBIOPUT_VSCREENINFO* will set the screen information. Some other ioctl commands include *FBIOGETCMAP, FBIOPAN_DISPLAY*, etc. You can find the list of commands in the *uapi/linux/fb.h* file in the kernel source.

Now, let's look at how to write a module that will work as a very minimal framebuffer driver, with only the required components.

We need a few header files in our module, and with the basic structure of a module our starting point becomes…

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/errno.h>
#include <linux/string.h>
#include <linux/mm.h>
#include <linux/slab.h>
#include <linux/delay.h>
```